

LINUX THEORY 4 ASSIGNMENT

NAME	SIVARAMAKRISHNAN R
ROLL NO	261100690026
BRANCH	Cyber Security

Hard link vs. soft link

Part A — Create the links

Create a directory named `link_lab` and navigate to it.

Create a file named `security.txt` containing the following text:

Linux File System Security Lab

Confidential Security Report

Create:

a hard link named `security_hard.txt`

a symbolic link named `security_soft.txt`

List all three files using a command that displays their inode numbers.

Record the inode numbers of:

`security.txt`

`security_hard.txt`

`Security_soft.txt`

```
siva@linux:~$ cd lab4
siva@linux:~/lab4$ echo -e "Linux File System Security Lab\nconfidentials Security Report" > security.txt
siva@linux:~/lab4$ ln security.txt security_hard.txt
siva@linux:~/lab4$ ln -li
error: unexpected argument '-l' found

    tip: to pass '-l' as a value, use '-- -l'

Usage: ln [OPTION]... [-T] TARGET LINK_NAME
      ln [OPTION]... TARGET
      ln [OPTION]... TARGET... DIRECTORY
      ln [OPTION]... -t DIRECTORY TARGET...

For more information, try '--help'.
siva@linux:~/lab4$ ls -li
total 16
667398 -rw-rw-r-- 1 siva siva    0 Sep 12 04:28 critical_alerts.txt
667384 -rw-rw-r-- 1 siva siva    0 Sep 12 04:26 eroor.txt
667397 -rw-rw-r-- 1 siva siva    0 Sep 12 04:27 error.txt
667393 -rw-rw-r-- 2 siva siva   61 Sep 13 13:04 security.txt
667393 -rw-rw-r-- 2 siva siva   61 Sep 13 13:04 security_hard.txt
667381 -rw-rw-r-- 1 siva siva 6960 Sep 11 17:21 security_log.txt
```

What difference do you observe?

security.txt and security_hard.txt share the same inode number, indicating that both names refer to the same file.

security_soft.txt has a different inode number because it is a separate file that simply stores the path to security.txt.

Part B — Examine File Metadata

Use `ls -l` to examine the three files. Identify the file type of each file from the first character of the permission field.

Compare the link count of the original file, hard link, and symbolic link. Explain the observed values.

```
siva@linux:~/lab4$ stat security_hard.txt
  File: security_hard.txt
  size: 61          Blocks: 8          IO Block: 4096   regular file
Device: 8,2    Inode: 667393    Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/   siva)   Gid: ( 1000/   siva)
Access: 2026-09-13 13:05:00.763769787 +0000
Modify: 2026-09-13 13:04:59.919769753 +0000
Change: 2026-09-13 13:05:25.430770767 +0000
Birth: 2026-09-13 13:04:59.918769752 +0000
siva@linux:~/lab4$ stat security_soft.txt
  File: security_soft.txt
  size: 0           Blocks: 0          IO Block: 4096   regular empty file
Device: 8,2    Inode: 667379    Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/   siva)   Gid: ( 1000/   siva)
Access: 2026-09-13 13:13:41.824960704 +0000
Modify: 2026-09-13 13:13:41.824960704 +0000
Change: 2026-09-13 13:13:41.824960704 +0000
Birth: 2026-09-13 13:13:41.824960704 +0000
siva@linux:~/lab4$
```

Use the `stat` command on all three files and record the following information:

Parameter	Original	Hard Link	Soft Link
Inode number	667393	667393	667379
File type	Regular file	Regular file	Symbolic link
Size	61	61	10
Link count	2	2	1
Permissions	0664	0664	0664

Compare the file sizes of the original file, hard link, and symbolic link. Why might the size of the symbolic link differ from the original file?

The original file and the hard link are both 61 bytes because they point to the same file. The soft link is only 10 bytes because it stores the name/path of the original file, not its actual contents.

File	First char	Type
security.txt	-	Regular file
security_hard.txt	-	Regular file
security_soft.txt		Symbolic link

Part C — Investigate readlink

Execute readlink on security_soft.txt. What information is displayed?

```
readlink security_soft.txt
```

The output is: security.txt

This shows that security_soft.txt is a symbolic link pointing to security.txt.

Execute readlink on security_hard.txt. What do you observe?

```
readlink security_hard.txt
```

Observation:

There is no output because security_hard.txt is a hard link. A hard link does not store the pathname of the original file; it directly refers to the same inode.

Based on your observations, explain why readlink works differently for hard links and symbolic links.

Use: readlink -f security_soft.txt

It displays the complete absolute path of the final target of the symbolic link, for example:
/home/aduuu/link_lab/security.txt

Explanation: readlink works differently because a symbolic link stores the pathname of its target, so readlink can display that pathname. A hard link does not store a target pathname; it directly refers to the same inode as the original file, so readlink security_hard.txt produces no output. readlink -f follows the symbolic link and resolves it to the final absolute path.

Use:

readlink -f security_soft.txt

What does the -f option reveal?

```
siva@linux:~/lab4$ readlink security_soft.txt
siva@linux:~/lab4$ readlink security_hard.txt
siva@linux:~/lab4$ readlink -f security_hard.txt
/home/siva/lab4/security_hard.txt
siva@linux:~/lab4$
```

Part D — Content Modification

Append the following line to security.txt:Unauthorized login detected

This appends the line to security.txt without deleting the existing content.

Display the contents using all three filenames.

cat security.txt cat

security_hard.txt

cat security_soft.txt

The line Unauthorized login detected appears in all three outputs, because the hard link and symbolic link both access the same file content.

Is the newly added content visible through both links? Explain why.

Modify the file using security_hard.txt. Check the contents through security.txt and security_soft.txt. What do you observe?

Yes. The newly added content is visible through both links.

security_hard.txt is a hard link to the same inode and file data as security.txt.

security_soft.txt is a symbolic link that points to security.txt.

Therefore, when content is added to security.txt, the updated content is accessible through both links.

```

siva@linux:~/lab4$ readlink security_hard.txt
siva@linux:~/lab4$ readlink -f security_hard.txt
/home/siva/lab4/security_hard.txt
siva@linux:~/lab4$ echo "Unauthorized login detected" >> security.txt
siva@linux:~/lab4$ cat security.txt
Linux File System Security Lab
confidentials Security Report
Unauthorized login detected
siva@linux:~/lab4$ cat security_hard.txt
Linux File System Security Lab
confidentials Security Report
Unauthorized login detected
siva@linux:~/lab4$ cat security_hard_soft.txt
cat: security_hard_soft.txt: No such file or directory
siva@linux:~/lab4$ cat security_soft.txt
siva@linux:~/lab4$ ln -s security.txt security_soft.txt
ln: Already exists
siva@linux:~/lab4$ cat security_soft.txt
siva@linux:~/lab4$ echo "System security alert" >> security_hard.txt
siva@linux:~/lab4$ cat security.txt
Linux File System Security Lab
confidentials Security Report
Unauthorized login detected
System security alert

```

Part E — Rename Experiment

Rename:

security.txt

to:

incident.txt

Try accessing the content through security_hard.txt. Does it still work?

`mv security.txt incident.txt`

`cat security_hard.txt`

It's working

Try accessing the content through security_soft.txt. Does it still work?

`cat security_soft.txt`

It's doesn't works

Use readlink security_soft.txt after renaming the original file. What path is still stored in the symbolic link?

It will display: **security.txt**

This is because the symbolic link stores the original pathname, even though the file was renamed to incident.txt.

Explain why renaming the original file affects the symbolic link differently from the hard link.

- **Hard link:** security_hard.txt still works and displays the file's contents. Hard links point directly to the inode, so renaming security.txt does not affect them.

- **Soft link:** If security_soft.txt points to security.txt, it stops working because the original pathname no longer exists. The link becomes broken.

```
siva@linux:~/lab4$ cat security_hard.txt
Linux File System Security Lab
confidentials Security Report
Unauthorized login detected
System security alert
siva@linux:~/lab4$ ls -l
total 16
-rw-rw-r-- 1 siva siva  0 Sep 12 04:28 critical_alerts.txt
-rw-rw-r-- 1 siva siva  0 Sep 12 04:26 eroor.txt
-rw-rw-r-- 1 siva siva  0 Sep 12 04:27 error.txt
-rw-rw-r-- 2 siva siva 111 Sep 13 13:22 security.txt
-rw-rw-r-- 2 siva siva 111 Sep 13 13:22 security_hard.txt
-rw-rw-r-- 1 siva siva 6960 Sep 11 17:21 security_log.txt
-rw-rw-r-- 1 siva siva  0 Sep 13 13:13 security_soft.txt
siva@linux:~/lab4$ cat security_hard.txt
Linux File System Security Lab
confidentials Security Report
Unauthorized login detected
System security alert
siva@linux:~/lab4$ cat security_soft.txt
```

Part F — Deletion Experiment

Recreate the links if necessary and then perform the following.

security_hard.txt

Delete the original file.

rm security.txt

Check whether security_hard.txt can still be accessed.

cat security_hard.txt

Yes, security_hard.txt can still be accessed and will display the file contents. This is because the hard link refers directly to the same inode, even after the original filename is deleted.

Check whether security_soft.txt can still be accessed.

cat security_soft.txt

No, security_soft.txt cannot be accessed. It will give a "No such file or directory" error because its target security.txt has been deleted. The symbolic link becomes a dangling/broken link.

Use ls -l and readlink to investigate the symbolic link after deleting its target.

```
ls -l security_soft.txt
```

```
readlink security_soft.txt
```

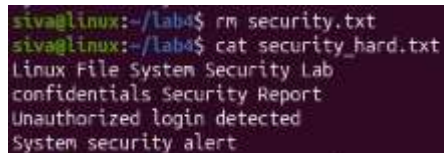
ls -l shows:

```
security_soft.txt -> security.txt
```

This shows the symbolic link still exists, but its target has been deleted.

readlink displays: security.txt

This confirms that the link still stores the old target path. security_soft.txt is now a dangling/broken symbolic link.



```
siva@linux:~/lab4$ rm security.txt
siva@linux:~/lab4$ cat security_hard.txt
Linux File System Security Lab
confidential: Security Report
Unauthorized login detected
System security alert
```

What is a dangling/broken symbolic link?

A dangling/broken symbolic link is a symbolic link whose target file or path no longer exists.

Does deleting the original filename immediately delete the underlying data when a hard link exists? Justify your answer using the link count and inode.

No. Deleting the original filename does not immediately delete the underlying data when a hard link exists.

Before deletion, the inode has a link count of 2 (security.txt and security_hard.txt).

After deleting security.txt, the link count becomes 1.

The same inode and underlying data remain, so security_hard.txt can still access the contents.

Part G — Directory and Filesystem Behaviour

Create a directory named evidence. Attempt to create a hard link to this directory. Record the result.

Expected result: ln: evidence: hard link not allowed for directory

The hard link creation fails because hard links to directories are not normally allowed.

Create a symbolic link named evidence_link pointing to the evidence directory. Was the operation successful?

```
ln -s evidence evidence_link
```

The symbolic link evidence_link is successfully created and points to the evidence directory. Symbolic links can point to directories.

Based on your experiment, compare hard and symbolic links with respect to directory linking.

Hard link: Cannot normally be created for a directory. The attempt fails.

Symbolic link: Can be created for a directory successfully. `evidence_link` points to the `evidence` directory.

If multiple filesystems/mount points are available in the laboratory environment, attempt to create:

a hard link across filesystems

Hard links cannot span different filesystems because they reference the same inode.

Therefore, attempting to create one across filesystems fails with an error such as “Invalid cross-device link”.

a symbolic link across filesystems

Symbolic links, however, can point to files on another filesystem because they store the target's pathname:

Parameter	Hard Link	Symbolic Link
Inode number	Same as target	Different from target
File type	Regular file	Symbolic link
Link count	Increases	Usually 1
File size	Same as target	Size of stored pathname
Stores target pathname	No	Yes
<code>readlink</code> output	No output	Shows target pathname
Effect of modifying target	Change is visible	Change is visible
Effect of renaming target	Still works	Becomes broken
Effect of deleting target	Still works if another hard link exists	Becomes broken
Can it become a dangling or broken link?	No	Yes
Directory linking	Not normally allowed	Allowed
Cross-filesystem linking	Not allowed	Allowed

Final Comparison

Based only on your experimental observations, complete the following table:

Parameter	Hard Link	Symbolic Link
-----------	-----------	---------------

Inode number		
--------------	--	--

File type		
-----------	--	--

Link count		
------------	--	--

File size		
-----------	--	--

Stores target pathname?		
-------------------------	--	--

readlink output		
-----------------	--	--

Effect of modifying target		
----------------------------	--	--

Effect of renaming target		
---------------------------	--	--

Effect of deleting target		
---------------------------	--	--

Can become a dangling or broken link?		
---------------------------------------	--	--

Directory linking		
-------------------	--	--

Cross-filesystem linking		
--------------------------	--	--

sort

Suppose alerts.txt contains:

192.168.1.10 admin 5

10.0.0.8 root 12

172.16.0.5 guest 2

10.0.0.20 admin 8

```
siva@linux:~/lab4$ nano alerts.txt
siva@linux:~/lab4$ sort -k3 -n alerts.txt

172.16.0.5 guest 2
192.168.1.10 admin 5
10.0.0.20 admin 8
10.0.0.8 root 12
siva@linux:~/lab4$
```

Format:

IP-address username failed attempts

Sort according to failed attempts

Sort a list of suspicious IP addresses alphabetically.

```
siva@linux:~/lab4$ nano suspicious_ips.txt
siva@linux:~/lab4$ sort suspicious_ips.txt
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10
siva@linux:~/lab4$ sort -u suspicious_ips.txt
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10
```

Display only unique suspicious IP addresses using sort.

```
siva@linux:~/lab4$ nano suspicious_ips.txt
siva@linux:~/lab4$ sort suspicious_ips.txt
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10
siva@linux:~/lab4$ sort -u suspicious_ips.txt
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10
```

Sort a list of detected port numbers numerically.

```
siva@linux:~/lab4$ nano ports.txt
siva@linux:~/lab4$ cat ports.txt
22
80
443
8080
21
siva@linux:~/lab4$ sort -n ports.txt
21
22
80
443
8080
```

Display port numbers in descending numerical order.

```
siva@linux:~/lab4$ sort -n ports.txt
21
Files
80
443
8080
siva@linux:~/lab4$ sort -nr ports.txt
8080
443
80
22
21
```

Sort usernames without considering uppercase/lowercase differences.

```
siva@linux:~/lab4$ nano usernames.txt
siva@linux:~/lab4$ sort -f usernames.txt
Alice
alice
BOB
bob
CHARLIE
charlie
```

From a failed-login log, count how many times each IP address occurs and display the most frequent IP first.

```
siva@linux:~/lab4$ nano failed_login.log
siva@linux:~/lab4$ sort failed_login.log | uniq -c | sort -nr
1 192.168.1.10
1 172.16.0.5
1 10.0.0.8
1 10.0.0.20
```

Given IP,username,failed_attempts, sort records based on the number of failed attempts.

```
siva@linux:~/lab4$ nano records.csv
siva@linux:~/lab4$ sort -t',' -k3 -n records.csv
10.0.0.20 admin 8
10.0.0.8 root 12
172.16.0.5 guest 2
192.168.1.10 admin 5
```

Sort a CSV vulnerability report based on severity score.

Extract IP addresses using grep, then use sort and uniq to identify unique IP addresses.

```
siva@linux:~/lab4$ grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' failed_login.log | sort -u
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10
```

Create a pipeline that displays the top 5 IP addresses responsible for the most failed login attempts.

```
siva@linux:~/lab4$ grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' failed_login.log | sort -u | uniq -c | sort -nr
1 192.168.1.10
1 172.16.0.5
1 10.0.0.8
1 10.0.0.20
```